TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 3: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions live in `starter_code.py` at the week root: `edges_to_adjacency_list`, `edges_to_adjacency_matrix`, `random_undirected` `random_dag`, and the `UnionFind` class (union by rank + path compression). Vertices are always labeled $0, 1, \ldots, n-1$.

The 18 problems are grouped into four themes:

- **Graph representations** (Problems 1–3): edge lists, adjacency lists/matrices, degrees, and the space/density trade-off.
- **BFS** (Problems 4–7): shortest-path distances, layers, path reconstruction, connected components, and diameter.
- **DFS** (Problems 8–14): recursive and iterative traversal, discovery/finish times, connected components, cycle detection (undirected and directed), and bipartiteness / odd-cycle certificates.
- **Topological sort & Union-Find** (Problems 15–18): DFS-based and Kahn's topological sort, counting topological orders, and Quick-Find.

1 Graph Representations

Problem 1 – Edge List to Adjacency List/Matrix

Implement two graph-representation conversions **from scratch** (without using the helpers of the same name in `starter_code.py` – this is a warm-up exercise):

- `edges_to_adjacency_list(n, edges, directed=False)`: return a list of length n, where `adj[u]` is the list of neighbors of u (in the order the edges were given). For undirected graphs, each edge (u, v) must appear in *both* `adj[u]` and `adj[v]`.
- `edges_to_adjacency_matrix(n, edges, directed=False)`: return an $n \times n$ matrix (list of lists) where `matrix[u][v]` is the *number* of edges between u and v (so parallel edges show up as values > 1).

Examples. For the undirected triangle 0-1, 1-2, 0-2: `adj[0]` contains $\{1, 2\}$, and the matrix is symmetric with `matrix[0][1]` = `matrix[1][0]` = 1. For directed edges $(0, 1), (1, 2), (0, 1)$ (a parallel edge $0 \to 1$): `adj[0] == [1,1]`, `matrix[0][1]` = 2 but `matrix[1][0]` = 0 (not symmetric). For an undirected self-loop $(0, 0)$: `adj[0]` contains 0 *twice* (degree convention), but `matrix[0][0]` = 1 (the matrix entry is incremented only once per edge, even for undirected self-loops, since there is only one such edge in the edge list).

Problem 2 – Degrees, Self-Loops, and Multi-Edges

Given an undirected graph as an edge list, implement:

- `degree_sequence(n, edges)`: return a list `deg` of length n where `deg[v]` is the degree of vertex v. A self-loop (u, u) contributes 2 to `deg[u]` (so the Handshake Lemma $\sum \texttt{deg} = 2|E|$ holds even with self-loops).
- `has_self_loop(edges)`: return `True` iff some edge (u, u) is present.
- `has_multi_edge(edges)`: return `True` iff some unordered pair $\{u, v\}$ $(u \neq v)$ appears more than once in `edges`.
- `handshake_check(n, edges)`: return `True` iff `sum(degree_sequence(n, edges)) == 2 * len(edges)` – a sanity check / illustration of the Handshake Lemma.

Problem 3 – Converting Between Adjacency Matrix and Adjacency List

Implement two conversions between the adjacency-matrix and adjacency-list representations of a *simple* undirected graph (no self-loops, no parallel edges, matrix entries are 0 or 1):

- `matrix_to_adjacency_list(matrix)`: given an $n \times n$ 0/1 matrix, return the adjacency list (list of length n, `adj[v]` sorted in increasing order).
- `adjacency_list_to_matrix(adj)`: given an adjacency list, return the corresponding $n \times n$ 0/1 matrix.

Also implement `matrix_density(matrix)`, returning the fraction of possible edges that are present: $\frac{\text{(number of 1-entries in the upper triangle)}}{n(n-1)/2}$, as a float. This illustrates the space trade-off between the two representations: an adjacency matrix uses $\Theta(n^2)$ space regardless of density, while an adjacency list uses $\Theta(n + m)$ space, which is much smaller for sparse graphs (low density).

2 Breadth-First Search

Problem 4 – BFS Distances and Layers

Implement `bfs_distances(adj, source)`, which runs BFS from `source` on a graph given as an adjacency list `adj` (the graph may be directed or undirected – BFS does not care). Return a list `dist` of length `len(adj)` where:

- `dist[source] == 0`
- `dist[v]` is the length (number of edges) of the *shortest* path from `source` to `v`, for every `v` reachable from `source`.
- `dist[v] == -1` for every `v` *not* reachable from `source`.

Also implement `bfs_layers(adj, source)`, which returns a list of "layers": `layers[0] == [source]`, `layers[1]` is all vertices at distance 1, etc. (within each layer, vertices appear in the order they were first discovered).

Problem 5 – Shortest Path Reconstruction via BFS

Implement `bfs_shortest_path(adj, source, target)`, which returns the actual sequence of vertices on a *shortest* path from `source` to `target` (unweighted graph), as a list `[source, ..., target]`.

If `target` is unreachable from `source`, return `None`. If `source == target`, return `[source]`.

Hint: run BFS while recording, for each vertex `v`, the vertex `parent[v]` from which `v` was first discovered (a "BFS tree"). Then walk `parent` pointers backward from `target` to `source` and reverse.

Problem 6 – Connected Components via BFS

Implement `connected_components_bfs(adj)`, which takes an adjacency list of an *undirected* graph with $n =$ `len(adj)` vertices and returns a list of components, where each component is a sorted list of vertices.

Components should be ordered by their smallest vertex (process vertices $0, 1, 2, \ldots$ in order, starting a new BFS whenever an unvisited vertex is found). Use BFS as the underlying traversal.

Also implement `num_components(adj)`, returning just the count.

Problem 7 – Graph Diameter via Repeated BFS

The *diameter* of a connected, unweighted graph is the maximum, over all pairs of vertices (u, v), of the shortest-path distance between u and v (the "longest shortest path").

Implement `eccentricity(adj, v)`, returning the maximum distance from `v` to any other vertex (using BFS), and `diameter(adj)`, returning the diameter of the graph by computing `eccentricity(adj, v)` for *every* vertex `v` and taking the overall maximum.

Assume `adj` represents a *connected* undirected graph. For a graph with a single vertex, the diameter is 0. This reuses `bfs_distances` from Problem 4.

3 Depth-First Search

Problem 8 – Recursive DFS Traversal and Discovery/Finish Times

Implement `dfs_recursive(adj, source)`, which performs a recursive DFS starting from `source` on a graph given as an adjacency list (directed or undirected) and returns the list of vertices in the order they are first visited (discovered), i.e. the DFS preorder. Visit neighbors of u in the order they appear in `adj[u]`.

Also implement `dfs_discovery_finish_times(adj, source)`, returning a pair of dicts `(discovery, finish)` mapping each *reachable* vertex to an integer "timestamp": start a counter at 0, increment it each time a vertex is discovered (recorded in `discovery`) or finished (recorded in `finish`, when the recursive call for that vertex returns). This is the classic discovery/finish-time bookkeeping used to classify edges (tree, back, forward, cross).

Problem 9 – Iterative DFS with an Explicit Stack

Implement `dfs_iterative(adj, source)`, an *iterative* version of DFS using an explicit stack (a Python list with `append`/`pop`) instead of recursion. Return the list of vertices in the order they are first visited (discovered), exactly like `dfs_recursive` from Problem 8.

To match the recursive version's order (which visits `adj[u][0]` before `adj[u][1]`, etc.), when pushing multiple neighbors of u at once, push them in *reverse* order, so the first neighbor is popped first.

Also implement `max_stack_size(adj, source)`, returning the maximum size the explicit stack reaches during the traversal – illustrating the $O(n)$ auxiliary space DFS can use in the worst case (e.g. a path graph).

Problem 10 – Connected Components via DFS

Implement `connected_components_dfs(adj)`, the DFS analogue of Problem 6: given an adjacency list of an *undirected* graph, return a list of components (each a sorted list of vertices), ordered by smallest vertex, using DFS instead of BFS.

Then implement `same_component(adj, u, v)`, returning `True` iff `u` and `v` are in the same connected component.

Finally, implement `is_connected(adj)`, returning `True` iff the whole graph is a single connected component (or has 0 or 1 vertices).

Note: for an undirected graph, the *set* of vertices visited from a given start does not depend on BFS vs. DFS – only the discovery *order* differs, so `connected_components_dfs` should return the same partition as `connected_components_bfs` from Problem 6 (implement DFS directly, without importing Problem 6).

Problem 11 – Cycle Detection in an Undirected Graph (DFS)

Implement `has_cycle_undirected(adj)`, which returns `True` iff the *undirected* graph given by adjacency list `adj` contains at least one cycle (in any connected component).

Use DFS: for each unvisited vertex, run a DFS keeping track of each vertex's *parent* in the DFS tree. If DFS encounters an edge to an already-visited vertex that is *not* the current vertex's parent, that is a "back edge" and indicates a cycle.

Important: in an adjacency list built from `edges_to_adjacency_list`, each undirected edge (u, v) appears as v in `adj[u]` *and* u in `adj[v]` – so when at u and seeing neighbor $v == parent[u]$,

that is just the edge arrived on, *not* a cycle. Only a visited, non-parent neighbor (or a self-loop) indicates a cycle.

Also implement `find_a_cycle(adj)`, which returns a list of vertices forming one cycle (e.g. `[a, b, c, a]` meaning the cycle a-b-c-a), or `None` if the graph is acyclic (a forest).

Problem 12 – Bipartiteness Testing via BFS 2-Coloring

A graph is *bipartite* iff its vertices can be partitioned into two sets A and B such that every edge has one endpoint in A and one in B (equivalently: it can be properly colored with 2 colors).

Implement `two_color_bfs(adj)`, which attempts to 2-color the graph using BFS: assign the source of each BFS a color (0), then alternate colors level by level (every neighbor gets the *opposite* color of its discoverer). Handle graphs with multiple connected components (run BFS from every unvisited vertex).

Return a list `color` of length `len(adj)` where `color[v]` $\in \{0, 1\}$ for every vertex, *if* the graph is bipartite. If the graph is *not* bipartite, return `None`.

Then implement `is_bipartite(adj)`, returning `True`/`False`.

Problem 13 – Finding an Odd Cycle (Certificate of Non-Bipartiteness)

Theorem (Kleinberg & Tardos). A graph is bipartite iff it contains no odd cycle. Problem 12 only tells you *whether* a graph is bipartite; this problem asks you to produce a *certificate* when it is not – an explicit odd cycle.

Implement `find_odd_cycle(adj)`:

- If the graph is bipartite, return `None`.
- Otherwise, return a list of vertices `[v0, v1, ..., vk, v0]` (first == last) forming a cycle of *odd* length $k + 1$ (i.e. $k + 1$ is odd, so there are an odd number of *edges* in the cycle).

Hint: run BFS 2-coloring (Problem 12) from each connected component's start vertex, tracking each vertex's BFS-tree parent and distance from the root. When you find an edge (u, v) where `color[u] == color[v]` (a "same-color edge"), the path root$\to u$, the edge u-v, and the path $v \to$root together form a cycle of length $dist[u] + dist[v] + 1$, which is odd exactly when $dist[u]$ and $dist[v]$ have the same parity (which they must, since u and v have the same color). Reconstruct the cycle by walking parent pointers from u and from v back to the root, then splicing the two paths together with the edge (u, v).

Problem 14 – Cycle Detection in a Directed Graph (DFS with Colors)

Implement `has_cycle_directed(adj)`, which returns `True` iff the *directed* graph given by adjacency list `adj` (`adj[u]` = out-neighbors of u) contains a directed cycle.

Use the classic 3-color DFS:

- **White** (0): not yet visited.
- **Gray** (1): currently on the DFS recursion stack (an ancestor of the current vertex).
- **Black** (2): fully finished (popped off the stack).

A directed graph has a cycle iff DFS ever encounters an edge to a *gray* vertex (a back edge to an ancestor). Edges to *black* vertices (forward or cross edges) do *not* indicate a cycle.

Also implement `find_a_directed_cycle(adj)`, returning a list of vertices `[v0, v1, ..., vk, v0]` forming a directed cycle (each consecutive pair, and the last→first pair, must be an edge in `adj`), or `None` if the graph is acyclic (a DAG).

4 Topological Sort & Union-Find

Problem 15 – Topological Sort via DFS

A *topological order* of a directed graph is an ordering of its vertices such that for every edge (u, v), u comes before v in the order. A directed graph has a topological order iff it is a DAG (no directed cycles).

Implement `topological_sort_dfs(adj)`:

- If `adj` (a directed graph) contains a cycle, return `None`.
- Otherwise, return a list of all vertices in a valid topological order.

Algorithm: run DFS from every unvisited vertex. When a vertex *finishes* (all its descendants fully explored), prepend it to the result list (or append and reverse at the end). This works because a vertex finishes only after all vertices reachable from it have finished, so it must come before them in topological order.

Implement `verify_topological_order(adj, order)`, returning `True` iff `order` is a permutation of `range(len(adj))` such that for every edge (u, v) in `adj`, u appears before v in `order`.

Problem 16 – Topological Sort via Kahn's Algorithm (BFS + In-Degree Queue)

Implement `topological_sort_kahn(adj)`, an alternative topological-sort algorithm using a queue (continuity with BFS / Problem 4):

1. Compute the in-degree of every vertex (number of incoming edges).
2. Initialize a queue with all vertices of in-degree 0.
3. Repeatedly dequeue a vertex u, append it to the result, and for each out-neighbor v of u, decrement v's in-degree; if it becomes 0, enqueue v.
4. If the result contains all n vertices, return it. Otherwise (some vertices never reached in-degree 0, because they're in a cycle), return `None`.

To make the result *deterministic* (for testing), use a `collections.deque` and, whenever multiple vertices simultaneously become available, enqueue them in increasing order of vertex index; initialize the queue with in-degree-0 vertices in increasing order too.

Also implement `in_degrees(adj)`, returning a list `indeg` of length `len(adj)` where `indeg[v]` is the number of edges (u, v) for any u.

Problem 17 – Counting Topological Orderings & Uniqueness

A DAG can have many valid topological orderings. This problem asks you to count them (for small DAGs) and relate uniqueness to the graph's structure.

Implement `count_topological_orders(adj)`, which returns the *total* number of distinct topological orderings of the DAG `adj`, using backtracking:

- At each step, the "available" vertices are those not yet placed whose in-degree (counting only edges from not-yet-placed vertices) is 0.
- Try placing each available vertex next, recurse, then undo.

- Base case: all vertices placed → count 1 ordering.

(For graphs where `adj` has a cycle, return 0 – there are no topological orderings.)

Then implement `has_unique_topological_order(adj)`, returning `True` iff `count_topological_orders(adj) == 1`.

Theorem connection: a DAG has a *unique* topological order iff it contains a Hamiltonian path (a path visiting every vertex exactly once) consistent with the edges – equivalently, iff every pair of "adjacent-in-the-order" vertices has a direct edge between them. **Example**: two independent chains $0 \to 1$ and $2 \to 3$ have $\binom{4}{2} = 6$ valid topological orderings (every interleaving that keeps 0 before 1 and 2 before 3); a single chain $0 \to 1 \to 2 \to 3$ has exactly 1.

Problem 18 – Quick-Find (Eager Union-Find)

Implement `QuickFind`, a disjoint-set data structure over elements $0, 1, \ldots, n-1$ using the *Quick-Find* strategy:

- Maintain an array `id_` of length n, where `id_[i]` is the "component id" (representative) of element i. Initially `id_[i] == i` for all i.
- `find(p)`: return `id_[p]` – $O(1)$.
- `union(p, q)`: if `find(p) != find(q)`, relabel *every* element whose id equals `id_[p]` to have id `id_[q]` instead – $O(n)$ per union.
- `connected(p, q)`: return `find(p) == find(q)` – $O(1)$.

Also track `self.find_calls` and `self.union_relabels` (total number of elements relabeled across all `union` calls), so the cost of Quick-Find can be measured empirically and compared against the $\Theta(n^2)$ worst case discussed in the lecture notes and theory questions (Quick-Union, union by rank, and path compression – the optimizations that bring this down to near-$O(1)$ amortized – are covered in the lecture notes and are already implemented for you in the general-purpose `UnionFind` class in `starter_code.py`, used by later weeks).